

Medical Image Classification Module

MedAssistAI

Milestone-3

Kaushika Yadav G

1. Introduction

Milestone 3 focused on integrating the machine learning components developed during the previous milestones with the MedAssistAI frontend and backend. The objective was to move from individual model development and experimentation towards a functional application workflow.

The trained disease prediction model was connected to the Flask backend, while the React frontend was updated to communicate with the backend and display the prediction results. This milestone established the basic end-to-end workflow between the user interface, backend services, and machine learning model.

2. Objectives

The main objectives of this milestone were:

- Integrate the trained machine learning model with the backend.
- Implement the disease prediction workflow through the Flask backend.
- Connect the frontend with the backend APIs.
- Enable users to submit symptoms for disease prediction.
- Display prediction results through the frontend.
- Integrate the developed components into the existing MedAssistAI application.
- Perform basic testing of the integrated workflow.

3. Machine Learning Integration

The trained disease prediction model was incorporated into the MedAssistAI application so that the model could be used for prediction during application runtime. Instead of retraining the model for every prediction request, the saved model files are loaded by the backend when required.

The integrated prediction module uses the trained model along with the required feature information and disease label encoder. The feature information ensures that the input symptoms are provided in the same structure expected by the model, while the label encoder converts the numerical prediction back into the corresponding disease name.

The following model-related files were integrated:

File	Purpose
best_decision_tree_model.pkl	Saved disease prediction model
disease_label_encoder.pkl	Converts encoded labels into disease names
feature_columns.pkl	Maintains the required symptom feature structure

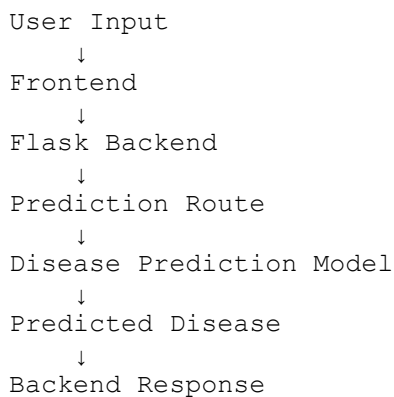
This allows the trained machine learning component to be reused by the application without performing the training process during runtime.

4. Backend Integration

The Flask backend was updated to support the machine learning prediction workflow. A prediction route was added to receive symptom information from the frontend and pass the processed information to the disease prediction module.

The backend acts as the intermediate layer between the frontend and the machine learning model. It receives the user's input, prepares the required data, invokes the prediction model, and returns the prediction result to the frontend.

Backend Workflow



5. Frontend Integration

The React frontend was updated to connect with the backend prediction services. The symptom analysis workflow allows the user to provide symptom information through the application interface.

The submitted information is sent to the backend through an API request. After the backend processes the request and obtains the model prediction, the result is returned to the frontend and displayed to the user.

This integration connects the user-facing interface with the machine learning functionality and allows the prediction system to be accessed through the MedAssistAI application.

6. End-to-End Workflow

The integrated workflow of the application can be summarized as follows:

1. The user logs into the MedAssistAI application.
2. The user navigates to the symptom analysis section.
3. The user provides the required symptoms.
4. The frontend sends the information to the Flask backend.
5. The backend processes the received symptoms.
6. The prediction module prepares the input according to the model's required features.
7. The trained model generates the disease prediction.
8. The disease label is converted into a readable disease name.
9. The backend returns the prediction to the frontend.

10. The frontend displays the prediction result to the user.

This establishes the basic connection between the **frontend** → **backend** → **machine learning model** → **frontend**.

7. Testing and Validation

Basic testing was performed after integrating the machine learning and application components. The backend prediction functionality was checked to ensure that the model could be loaded and used for inference.

The frontend and backend communication was also tested to verify that symptom information could be submitted successfully and that the prediction response could be received and displayed by the frontend.

The integration process helped identify issues related to data formatting, API communication, and model input compatibility. These were addressed as part of the integration process.

8. Outcomes

The major outcomes of Milestone 3 were:

- Machine learning functionality was integrated with the Flask backend.
- Disease prediction API workflow was established.
- Saved model and supporting files were added to the project.
- Frontend symptom analysis was connected to the backend.
- Prediction results could be returned to the frontend.
- Frontend, backend, and machine learning components were organized into a unified application.
- An end-to-end disease prediction workflow was established.

9. Future Work

The next phase will focus on improving the prediction performance and reliability of the machine learning component. Additional models can be trained and compared to identify the most suitable model for the disease prediction workflow.

Further development will also focus on improving disease probability reporting, risk assessment, severity analysis, system testing, and deployment of the complete MedAssistAI application.

10. Conclusion

Milestone 3 focused on integrating the machine learning and application components developed by the team. The disease prediction model was connected to the Flask backend, and the React frontend was connected to the backend prediction workflow.

The milestone established the basic end-to-end functionality of MedAssistAI, allowing user-provided symptoms to be processed by the backend and passed to the machine learning model for prediction. This provides the foundation for further model improvement, risk assessment, and deployment in subsequent stages.